



UNIVERSIDADE FEDERAL DO ESTADO DO RIO DE JANEIRO  
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA  
ESCOLA DE INFORMÁTICA APLICADA

# Relatório do Trabalho Final de Estrutura de Dados: Ordenação Topológica

RAUL CANDIDO DE SOUSA RODRIGUES  
RHUAN SOARES RAMOS  
THAUAN FABRÍCIO DA ROCHA

RIO DE JANEIRO, RJ – BRASIL  
JULHO DE 2025

# Sumário

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>Sumário.....</b>                              | <b>2</b>  |
| <b>1.Introdução.....</b>                         | <b>3</b>  |
| 1.1 Configuração do sistema.....                 | 3         |
| 1.2 IDE, linguagem, classes e métodos.....       | 3         |
| <b>2. Ordenação Topológica.....</b>              | <b>4</b>  |
| 2.1. Classe interna: Elo.....                    | 4         |
| 2.2. Classe interna: EloSuc.....                 | 4         |
| 2.3. Método Público: realizaLeitura.....         | 5         |
| 2.3.1. Método Privado: realizaLeituraAux.....    | 5         |
| 2.3.2. Método: buscarElo.....                    | 6         |
| 2.3.3. Método: atualizarDados.....               | 7         |
| 2.3.4. Método: insereFinal.....                  | 7         |
| 2.4. Método: debug.....                          | 8         |
| 2.4.1. Método: imprimeEloSuc.....                | 8         |
| 2.5. Método: buscaEloSemPred.....                | 9         |
| 2.6. Método: gerarSaida.....                     | 9         |
| 2.7. Método: isParcialmenteOrdenado.....         | 11        |
| 2.8. Método: executa.....                        | 12        |
| <b>3.Geração do Grafo.....</b>                   | <b>12</b> |
| 3.1. Método: gerarGrafo.....                     | 12        |
| <b>4. Main.....</b>                              | <b>13</b> |
| 4.1. Método: main.....                           | 14        |
| <b>5. Realização dos Testes.....</b>             | <b>15</b> |
| 5.1. Entradas.....                               | 15        |
| 5.2. Ordenação dos grafos.....                   | 15        |
| <b>6. Resultados.....</b>                        | <b>15</b> |
| 6.1. Gráfico baseado nos resultados obtidos..... | 17        |
| 6.2 Análise dos Gráficos:.....                   | 17        |
| <b>7. Conclusão.....</b>                         | <b>18</b> |
| <b>8. Referências.....</b>                       | <b>19</b> |

# 1.Introdução

Este trabalho tem como objetivo gerar grafos direcionados acíclicos (DAGs) com  $n$  vértices e utilizar ordenação topológica para produzir uma ordenação linear compatível com a ordem parcial inerente ao DAG.

A ordenação topológica desempenha um papel crucial em múltiplos domínios acadêmicos e práticos, tais como a programação de tarefas, a compilação de linguagens de programação e a modelagem de estruturas de dependência. Esta técnica proporciona uma sequencialização linear dos vértices em um grafo acíclico direto, assegurando que cada nó anteceda todos os seus respectivos descendentes na cadeia sucessória. Tal disposição preserva rigorosamente as relações de precedência estabelecidas, prevenindo qualquer infração à sequência hierárquica predefinida.

Neste trabalho, construímos um algoritmo de ordenação topológica e um algoritmo para geração de grafos acíclicos. Posteriormente, realizamos testes para verificar a complexidade do algoritmo de ordenação topológica e compará-la com a complexidade teórica esperada.

## 1.1 Configuração do sistema

As configurações do sistema utilizado para realizar esse estudo foram as seguintes:

### Hardware

- Lenovo ThinkPad E14 (1ª Geração)
- Processador - Intel® Core™ i5-10210U (10ª geração)
  - Arquitetura: Comet Lake (14nm)
  - Núcleos: 4 núcleos / 8 threads
  - Frequência: Base: 1.6 GHz Turbo: até 4.2 GHz
- Memória RAM: 16GB DDR4 (Single Channel)
- Armazenamento: SSD NVMe 256GB
- GPU Intel® UHD Graphics 620

### Software

- Sistema Operacional Windows 11 Pro 64-bit (23H2)
- IDE Eclipse IDE 2025-06 (4.32.0)
- JDK Oracle JDK 24
- Java Runtime Java SE 24 (build 24+36)

## 1.2 IDE, linguagem, classes e métodos

Foi utilizada a linguagem de programação Java e a Eclipse IDE para a implementação dos algoritmos propostos. Além disso, para a organização do código foram usadas três classes: Main.java, OrdenacaoTopologica.java e GeradorGrafos.java. A descrição de cada classe com seus métodos e atributos será descrita ao decorrer do trabalho.

## 2. Ordenação Topológica

A classe de OrdenacaoTopologica.java utilizada para implementar o algoritmo de Ordenação Topológica possui as seguintes classes internas: Elo e Elosuc.

Os atributos desta classe são:

Private Elo prim: referência para o primeiro Elo da lista encadeada.

Private int n: número de elementos na lista.

Esta classe possui o seguinte construtor:

Public OrdenacaoTopologica(): inicializa uma lista vazia e sem elementos.

Os métodos serão descritos após a explicação das classes internas.

### 2.1. Classe interna: Elo

Nesta seção será explicada a classe interna Elo. É uma classe privada interna que representa um nó na lista encadeada. Esta classe possui os seguintes atributos:

Public int chave: é utilizado para identificar um elemento na lista encadeada.

Public int contador: é utilizado para saber o número de predecessores de um nó.

Public Elo prox: é uma referência para o próximo elemento da lista encadeada.

Public EloSuc listaSuc: é uma referência para o primeiro elemento da lista de sucessores.

Esta classe possui os seguintes construtores:

Public Elo(): Inicializa um nó com zero predecessores, sem uma referência para o próximo elemento da lista encadeada e sem uma lista de sucessores.

Public Elo(int chave): Inicializa um nó com a chave passada por parâmetro, sem uma referência para o próximo elemento da lista encadeada e sem uma lista de sucessores.

Public Elo(int chave, int contador, Elo prox, EloSuc listaSuc): Inicializa um nó com a chave, número de predecessores, uma referência para o próximo da lista e uma referência para o primeiro elemento da lista de sucessores, tudo passado por parâmetro.

Esta classe possui os seguintes métodos:

Public String toString(): retorna uma string com a chave do Elo.

### 2.2. Classe interna: EloSuc

Nesta seção será explicada a classe interna EloSuc. É uma classe privada interna que representa um nó na lista de sucessores. Esta classe possui os seguintes atributos:

Public Elo id: é uma referência para o elo é sucessor.

Public EloSuc prox: é uma referência para o próximo elemento da lista encadeada.

Esta classe possui os seguintes construtores:

Public EloSuc(): instancia um elo vazio.

Public EloSuc(Elo id): instancia um elo com id passado por parâmetro.

Public EloSuc(Elo id, EloSuc prox): instancia um elo com id e prox passados por parâmetro.

Esta classe possui os seguintes métodos:

Public String toString(): retorna uma string com a chave do Elo.

## 2.3. Método Público: realizaLeitura

O método realizaLeitura tem como objetivo realizar a leitura do arquivo de entrada. Ele utiliza de uma lógica recursiva com o uso do método privado realizaLeituraAux citado no próximo subtópico.

Método Público:

```
public void realizaLeitura(String nomeEntrada) {  
    File arquivo = new File("src/entradas/"+nomeEntrada+".txt");  
    Scanner in = null;  
  
    try {  
        in = new Scanner(arquivo);  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
  
    while(in.hasNextLine()) {  
        realizaLeituraAux(in.nextLine());  
    }  
}
```

Complexidade do método:

O método em questão possui complexidade  $O(n^2)$ , em que  $n$  é número de linhas do arquivo, já que o método lê cada linha do arquivo. Ele é quadrático devido ao método recursivo realizaLeituraAux que é composto pelos métodos buscarElo, atualizarDados e insereFinal, em que suas complexidades são respectivamente  $O(n)$ ,  $O(1)$  e  $O(n)$ , em que serão explicadas nos subtópicos abaixo, resultando em  $O(n + 1 + n)$  que é possível descrever como  $O(n)$ . Logo temos  $n$  do método público e  $n$  do privado, assim tendo  $O(n^2)$ .

### 2.3.1. Método Privado: realizaLeituraAux

Método recursivo que irá realizar a leitura do conteúdo do arquivo. De forma que leia cada linha da forma 'x < y'.

Este método utiliza de três métodos auxiliares, que serão explicados logo abaixo, para sua realização sendo eles:

- buscarElo

- atualizarDados
- insereFinal

Método Privado:

```
private void realizaLeituraAux(String parChave)
{
    // x < y
    String[] entrada = parChave.split("<");

    int x = Integer.valueOf(entrada[0].trim());
    int y = Integer.valueOf(entrada[1].trim());

    Elo elox = buscarElo(prim,x);
    Elo eloy = buscarElo(prim,y);

    // Se x não existir
    if(elox == null) {
        elox = insereFinal(prim,x);
    }

    // Se y não nao existir
    if(eloy == null ) {
        eloy = insereFinal(prim,y);
    }

    atualizarDados(elox,eloy);
}
```

Complexidade do método:

O método possui complexidade  $O(n)$  decorrente dos métodos auxiliares que são utilizados, pois além dos métodos, ele possui apenas atribuições. Dos métodos auxiliares em questão temos buscarElo, atualizarDados e insereFinal, em que suas complexidades são respectivamente  $O(n)$ ,  $O(1)$  e  $O(n)$ , em que serão explicadas nos subtópicos abaixo, resultando em  $O(n + 1 + n)$  que é possível descrever como  $O(n)$ .

### 2.3.2. Método: buscarElo

Método responsável por buscar e retornar um Elo, para ser utilizado para as buscas e retorno dos elos x e y, utilizados no método privado de realizaLeituraAux.

Método:

```
private Elo buscarElo(Elo p,int x){

    while(p!=null) {
        if(p.chave == x)
            return p;
    }
```

```

        p = p.prox;
    }
    return null;
}

```

Complexidade:

O método possui complexidade  $O(n)$ , onde irá percorrer toda a lista.

### 2.3.3. Método: atualizarDados

Método que realiza a atualização dos elos x e y. Tendo um caso para verificar se é a primeira inserção dos elos em questão e outro para as demais inserções.

Método:

```

private void atualizarDados(Elo x, Elo y) {
    // Atualizando dados de X
    if(x.listaSuc == null) { // Primeira inserção
        x.listaSuc = new EloSuc(y);
    }
    else { // Demais inserções
        EloSuc aux = x.listaSuc; // Primeiro sucessor da lista
        x.listaSuc = new EloSuc(y); // Insere o novo elemento no inicio
        x.listaSuc.prox = aux; // Faz o novo elemento apontar para o
        antigo primeiro da lista
    }
    y.contador++; // Atualização de dados do y
}

```

Complexidade:

O método possui complexidade  $O(1)$ , pois apenas possui inserções e atribuições.

### 2.3.4. Método: insereFinal

Método responsável pela inserção do Elo no final da lista e pela incrementação do contador 'n' da classe de OrdemTopologica.

Método:

```

private Elo insereFinal(Elo ult,int x) {
    if(ult == null) { // Primeira inserção
        prim = new Elo(x); // Inicializa o Elo com a chave sendo x
        n++;
        return prim;
    }

    while(ult.prox != null)

```

```
ult = ult.prox;
```

```
Elo p = new Elo(x);  
ult.prox = p;  
n++;  
return p;  
  
}
```

Complexidade:

O método possui complexidade  $O(n)$ , pois terá que percorrer a lista até o final.

## 2.4. Método: debug

O método debug é o responsável pela impressão do estado atual da estrutura de dados, no modelo:  
2 predecessores: 1, sucessores: 10 -> 4 -> NULL.

O método utiliza o método auxiliar imprimeEloSuc para imprimir os Elo sucessores do método específico em questão.

Método:

```
private void debug()  
{  
    System.out.println("Debug");  
  
    Elo p = prim;  
    while (p != null) {  
        System.out.print(p + " predecessores: " + p.contador + ", sucessores: ");  
        imprimeEloSuc(p.listaSuc);  
        p = p.prox;  
    }  
}
```

Complexidade:

O método possui a complexidade  $O(n + s)$  em que  $n$  é devido a quantidade de vezes que imprime e  $s$  devido a número de sucessores.

### 2.4.1. Método: imprimeEloSuc

Método para imprimir o(s) Elo(s) sucessores do Elo em questão passado pelo parâmetro.

Método:

```
private void imprimeEloSuc(EloSuc p) {  
  
    while (p != null) {  
        System.out.print(p.id + " -> ");
```

```

        p = p.prox;
    }
    System.out.println("NULL");
}

```

Complexidade:

O método possui complexidade  $O(s)$ , onde  $s$  é o número de sucessores do Elo em questão

## 2.5. Método: buscaEloSemPred

Método para buscar os Elos sem predecessores.

Método:

```

private void buscaEloSemPred() {
    Elo p = prim; // Armazena a lista antiga
    prim = null; // Utiliza a lista corrente para armazenar a lista ordenada
    topologicamente

```

```

        while(p != null) {
            Elo q = p;
            p = q.prox;
            if(q.contador == 0) {
                // Insere q na nova Cadeia no começo
                q.prox = prim;

                //Atualiza o q como primeiro da lista pois coloca o novo elemento
                no início.

                prim = q;
            }
        }
    }
}

```

Complexidade:

O método possui complexidade  $O(n)$ , pois terá que percorrer a lista toda para buscar Elo sem predecessores

## 2.6. Método: gerarSaida

O método gerarSaida é o responsável por gerar a saída da ordenação topológica

Método:

```

private void gerarSaida() {

    // Utilizando um elo auxiliar para guardar referência ao começo da lista
    Elo q = prim;

```

```

//Utilizando um elo auxiliar para guardar o ultimo da lista
Elo ult = null;

//Verificando se a lista está vazia
if(prim != null) {

    // Usando a referência do primeiro para chegar no final da lista.
    ult = prim;

    // Guardando referência ao último elemento da lista.
    while(ult.prox != null) {
        ult = ult.prox;
    }
}

//Imprimindo de acordo com a saída esperada no enunciado.
System.out.println("Ordenação Topológica:");

//Estrutura repetitiva responsável por percorrer a lista com zero predecessores.
while(q != null) {

    //Imprimindo chave corrente
    System.out.print(q.chave + " ");

    //Decrementando o número de elementos na lista.
    n--;

    //Referência do começo da lista sendo passada para o próximo da lista,
    descartando o elemento q corrente.
    prim = q.prox;

    //Estrutura repetitiva utilizada para percorrer a lista sucessiva.
    for(EloSuc t = q.listaSuc; t != null; t = t.prox) {

        //Decrementando o número de predecessores dos elementos da lista
        sucessiva.

        t.id.contador--;

        //Verificando se o número de predecessores é 0.
        if(t.id.contador == 0) {

            //Verificando se a lista está vazia.
            if (prim == null) {

                //Se a lista estiver vazia, o próximo elemento da lista
                sucessiva vai ser o primeiro da lista com zero predecessores.
                prim = t.id;
            }
        }
    }
}

```

```

        ult = t.id;
        //Garante que o nó não traga outra lista com ele, causando
        um encadeamento indesejado.
        t.id.prox = null;
    }

    // Se a lista não estiver vazia.
    else {

        // O(1) para colocar no final da lista.
        ult.prox = t.id;

        // Atualizando o elo auxiliar para apontar para o último da
        lista.

        ult = t.id;

        //Garante que o nó não traga outra lista com ele, causando
        um encadeamento indesejado.
        t.id.prox = null;
    }

    //Eliminando elemento com 0 predecessores da lista de sucessores
    q.listaSuc = q.listaSuc.prox;
}
}
// Indo para o próximo elemento da list, descartando o anterior.
q = prim;
}
}

```

Complexidade:

O método possui complexidade  $O(n + m)$ , pois temos o loop externo que irá percorrer a lista  $n$  vezes e um loop interno que irá percorrer a lista de sucessores  $m$  vezes.

## 2.7. Método: isParcialmenteOrdenado

Método responsável por verificar se é parcialmente ordenado

Método:

```

private boolean isParcialmenteOrdenado() {
    return this.n == 0;
}

```

Complexidade:

O método possui complexidade  $O(1)$ , pois apenas apresenta atribuição.

## 2.8. Método: executa

Método responsável por executar o algoritmo chamando os métodos:

- debug
- buscaEloSemPred
- gerarSaida
- isParcialmenteOrdenado

Método:

```
public boolean executa()
{
    debug();
    buscaEloSemPred();
    gerarSaida();
    System.out.println();

    return isParcialmenteOrdenado();
}
```

Complexidade:  $O(n + m)$ , onde  $n$  é o número de vértices e  $m$  o número de arestas.

## 3. Geração do Grafo

A classe GeradorGrafos.java é utilizada no algoritmo para gerar as entradas que vão ser lidas pelo método realizarLeitura(String nomeEntrada). A classe possui só um método estático para facilitar a geração dos grafos de acordo com o número de vértices passados por parâmetro.

### 3.1. Método: gerarGrafo

Este método é responsável por gerar os grafos acíclicos.

Método:

```
public static void gerarGrafo(int nVertices) throws IOException {
    int maxArestas = nVertices * (nVertices - 1) / 2;
    Random rand = new Random();

    try (BufferedWriter escritor = new BufferedWriter(new
    FileWriter("src/entradas/"+nVertices + ".txt"))) {
        // 1. Embaralha vértices com Fisher-Yates
        int[] vertices = new int[nVertices];
        for (int i = 0; i < nVertices; i++) vertices[i] = i + 1;

        for (int i = nVertices - 1; i > 0; i--) {
            int j = rand.nextInt(i + 1);
            int temp = vertices[i];
```

```

        vertices[i] = vertices[j];
        vertices[j] = temp;
    }

    // 2. Gera arestas diretamente em vetores
    int[] x = new int[maxArestas];
    int[] y = new int[maxArestas];
    int idx = 0;

    for (int i = 0; i < nVertices; i++) {
        for (int j = i + 1; j < nVertices; j++) {
            x[idx] = vertices[i];
            y[idx] = vertices[j];
            idx++;
        }
    }

    // 3. Embaralha arestas com Fisher-Yates
    for (int i = maxArestas - 1; i > 0; i--) {
        int j = rand.nextInt(i + 1);
        // Troca x
        int tempX = x[i];
        x[i] = x[j];
        x[j] = tempX;
        // Troca y
        int tempY = y[i];
        y[i] = y[j];
        y[j] = tempY;
    }

    // 4. Escreve no arquivo
    for (int i = 0; i < maxArestas; i++) {
        escritor.write(x[i] + " < " + y[i] + "\n");
    }
}
}

```

Complexidade:  $O(n^2)$ , onde  $n = nVertices$ .

## 4. Main

A classe Main.java foi a responsável por executar o algoritmo. Ela possui só um método main(). Esta classe importa os seguintes packages: [java.io](#).BufferedWriter; [java.io](#).FileWriter; [java.io](#).IOException; java.util.Scanner.

## 4.1. Método: main

Este método é responsável pela realização dos testes.

Método:

```
public class Main
{
    public static void main(String args[]) throws IOException
    {
        Scanner in = new Scanner(System.in);

        OrdenacaoTopologica ord = new OrdenacaoTopologica();

        int grafo = in.nextInt();

        GeradorGrafos.gerarGrafo(grafo);

        String entrada = Integer.toString(grafo);

        long t0 = System.currentTimeMillis();
        for(int j = 0; j < 10; j++) {
            ord.realizaLeitura(entrada);
            if(!ord.executa()) {
                System.out.println("O conjunto nao é parcialmente
ordenado.");
            } else {
                System.out.println("O conjunto é parcialmente
ordenado.");
            }
        }
        long t1 = System.currentTimeMillis();

        long total = (t1-t0)/10;

        String linhaLog = "Grafo de " + entrada + " vértices executado em " + total + " ms";

        try (BufferedWriter writer = new BufferedWriter(new
FileWriter("src/br/unirio/resultados.txt", true))) {
            writer.write(linhaLog);
            writer.newLine();
        } catch (IOException e) {
            e.printStackTrace();
        }
        System.out.println(linhaLog);
    }
}
```

Complexidade:  $O(n^3)$ , onde  $n$  é o número de vértices do grafo fornecido como entrada.

## 5. Realização dos Testes

A realização dos testes ocorreu a partir de duas etapas que vão ser explicadas nesta seção. Todos os testes foram realizados na máquina dita no começo do relatório e também a linguagem utilizada quanto a IDE responsável por implementar e executar o algoritmo.

### 5.1. Entradas

Como foi dito, as entradas foram os grafos gerados pelo método `gerarGrafo()`, que tem como parâmetro a quantidade de vértices. A quantidade de vértices dos grafos gerados para a realização dos testes foram 10, 20, 30, 40, 50, 100, 200, 500, 1000, 5000, 10000 e 20000. É importante salientar que 20000 vértices foi o limite da máquina que utilizamos para realizar os testes, por isso não foram gerados grafos com mais vértices. O limite da memória impossibilitou a geração de grafos com mais vértices.

Os grafos foram salvos em arquivos `.txt` que foram lidos pelo método `realizarLeitura()`. Foi escolhido fazer desta forma para ter os grafos salvos e priorizar os testes da ordenação topológica. Nesse sentido, os grafos foram salvos da seguinte maneira “ $x < y$ ”, no qual  $x$  é anterior a  $y$  na ordenação.

### 5.2. Ordenação dos grafos

Após a criação dos grafos e a leitura deles, o método `buscaEloSemPred()` e logo em seguida o método `gerarSaida()` são chamados no `executa()` e verifica se o grafo é parcialmente ordenado. Antes de ir para a próxima seção, é fundamental relatar que foi levada em consideração a Lei dos Grandes Números para a realização dos testes. Por isso, cada grafo foi executado dez vezes e foi feita uma média na qual serão vistas e explicadas na seção de resultados.

## 6. Resultados

Os resultados experimentais de tempo de leitura e execução, medidos em segundos, foram adquiridos após a execução do algoritmo para um conjunto de grafos com diferentes números de vértices ( $n$ ).

Para analisar este comportamento, o desempenho experimental foi comparado a um modelo teórico de complexidade cúbica  $O(n^3)$  que representa a complexidade da leitura e da execução do algoritmo de ordenação topológica implementado. Para criar uma régua teórica de comparação relevante, a curva de  $O(n^3)$  foi normalizada. Foi selecionado o ponto experimental para  **$n = 1000$  vértices** (com tempo de 2.404 segundos) como âncora. A constante de proporcionalidade  $C$  foi calculada para que a curva teórica passasse exatamente por este ponto de referência, usando a definição assintótica:

$$C = \frac{T(n)}{f(n)} = \frac{2.404}{1000^3} \approx 2.404 \times 10^{-9}$$

Onde:

$T(n)$  = tempo de execução

$n$  = tamanho da entrada (número de vértices)

$c$  = constante de proporcionalidade (ambiente de execução)

Os dados obtidos estão na tabela a seguir:

| Número de Vértices ( $n$ ) | Tempo de Execução Experimental (segundos) | Tempo Teórico $O(n^3)$      | Tempo Teórico $O(n^2)$   |
|----------------------------|-------------------------------------------|-----------------------------|--------------------------|
| 10                         | 0.003                                     | 0.000002                    | 0.0002                   |
| 20                         | 0.002                                     | 0.000019                    | 0.0010                   |
| 30                         | 0.003                                     | 0.000065                    | 0.0022                   |
| 40                         | 0.003                                     | 0.000154                    | 0.0038                   |
| 50                         | 0.005                                     | 0.000301                    | 0.0060                   |
| 100                        | 0.012                                     | 0.002404                    | 0.0240                   |
| 200                        | 0.035                                     | 0.019232                    | 0.0962                   |
| 500                        | 0.3                                       | 0.300500                    | 0.6010                   |
| 1000                       | 2.404                                     | 2.404000                    | 2.4040                   |
| 5000                       | 190.357 $\approx$ 3 min                   | 300.500 $\approx$ 5 min     | 60.100 $\approx$ 1 min   |
| 10000                      | 1439.578 $\approx$ 24 min                 | 2404.000 $\approx$ 40 min   | 240.400 $\approx$ 4 min  |
| 20000                      | 10953.433 $\approx$ 182 min               | 19232.000 $\approx$ 321 min | 961.600 $\approx$ 16 min |

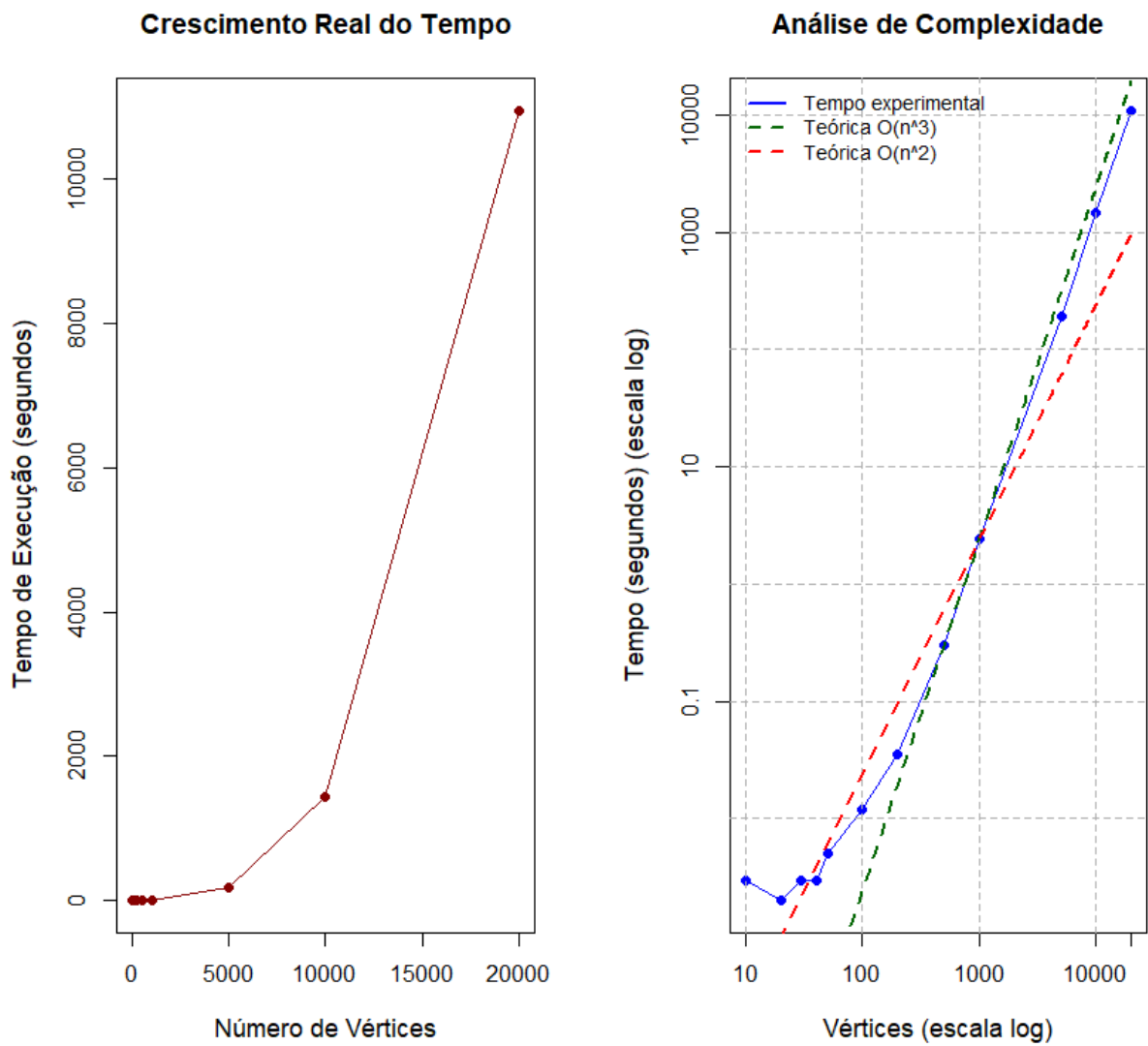
Para quantificar o expoente de complexidade prática do algoritmo para entradas  $> 500$ , aplicamos regressão linear em escala logarítmica utilizando o seguinte algoritmo em python:

```
import numpy as np
n = np.array([500, 1000, 5000, 10000, 20000])
t = np.array([0.300, 2.404, 190.357, 1439.578, 10953.433])
k = np.polyfit(np.log10(n), np.log10(t), 1)[0] # Coeficiente angular da reta
```

e obtivemos o resultado de  $k \approx 2.814$

## 6.1. Gráfico baseado nos resultados obtidos

Os dados foram analisados e plotados no R Studio, gerando gráficos comparativos entre resultados experimentais e teóricos.



## 6.2 Análise dos Gráficos:

**Gráfico da Esquerda (Crescimento Real):** Apresenta os dados em escala linear. A curva ascendente e acentuada confirma visualmente que o tempo de execução cresce de forma super-linear, característico de uma complexidade polinomial.

**Gráfico da Direita (Análise em Escala Log-Log):** Apresenta os dados em escala logarítmica, técnica essencial para analisar a ordem da complexidade. Este gráfico compara o desempenho experimental (linha azul) com a curva teórica de referência  $O(n^3)$  (linha verde)

tracejada) e  $O(n^2)$  (linha vermelha tracejada) marcados pela ancoragem do ponto **1000**. A partir dele, observamos dois pontos:

- Para entradas  $< 500$  vértices, o comportamento é mais ruidoso e menos linear, com complexidade aparentemente menor devido à dominância de termos de ordem inferior se aproximando mais da curva teórica de  $O(n^2)$ .
- Para entradas  $> 500$  vértices, o comportamento experimental mostra um expoente de entre 2 e 3, indicando uma complexidade intermediária entre  $O(n^2)$  e  $O(n^3)$ . Os dados permanecem consistentemente abaixo da previsão  $O(n^3)$  pura, revelando que fatores de implementação amorteceram o crescimento cúbico teórico. Essa convergência é particularmente evidente nos pontos de  $n=1000$ ,  $5000$ ,  $10000$  e  $20000$  vértices, onde os tempos experimentais seguem com um expoente estimado de  $O(n^{2.8})$ .

## 7. Conclusão

Diante dos resultados apresentados neste trabalho, após a implementação e testes dos algoritmos, verifica-se que o tempo experimental de execução aproxima-se significativamente da complexidade teórica esperada. A análise dos dados confirma um crescimento polinomial do tempo de execução, com comportamento consideravelmente abaixo de  $O(n^3)$  e com um expoente estimado de  $O(n^{2.8})$  para entradas maiores. Além disso, a ordenação linear dos vértices pelo algoritmo está em conformidade com o DAG, evidenciando que os grafos gerados são acíclicos e respeitam as normas estabelecidas. Os testes foram realizados até o limite de 20.000 vértices, pois a execução com o número de vértices superiores a essa medida resultou em problemas relacionados à capacidade de memória. Conclui-se, portanto, que  $O(n^3)$  representa um **modelo teórico coerente como limite superior**, ainda que o comportamento real se beneficie de otimizações internas que amortecem ligeiramente esse crescimento.

## 8. Referências

ESTEVEES, Raul. Algoritmo de Fisher-Yates para embaralhamento de arrays. Medium, [S. l.], 3 de Julho de 2018. Disponível em: <https://raullestev.es.medium.com/algoritmo-de-fisher-yates-para-embaralhamento-de-arrays-ba13a0542e88>

CARVALHO, Marco Antonio M. BCC204 - Teoria dos Grafos. (Baseado nas notas de aula do prof. Haroldo Gambini Santos). Departamento de Computação, Instituto de Ciências Exatas e Biológicas, Universidade Federal de Ouro Preto, 22 de outubro de 2019.

NUMPY. numpy.polyfit documentation. 2024. Disponível em: <https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html>.